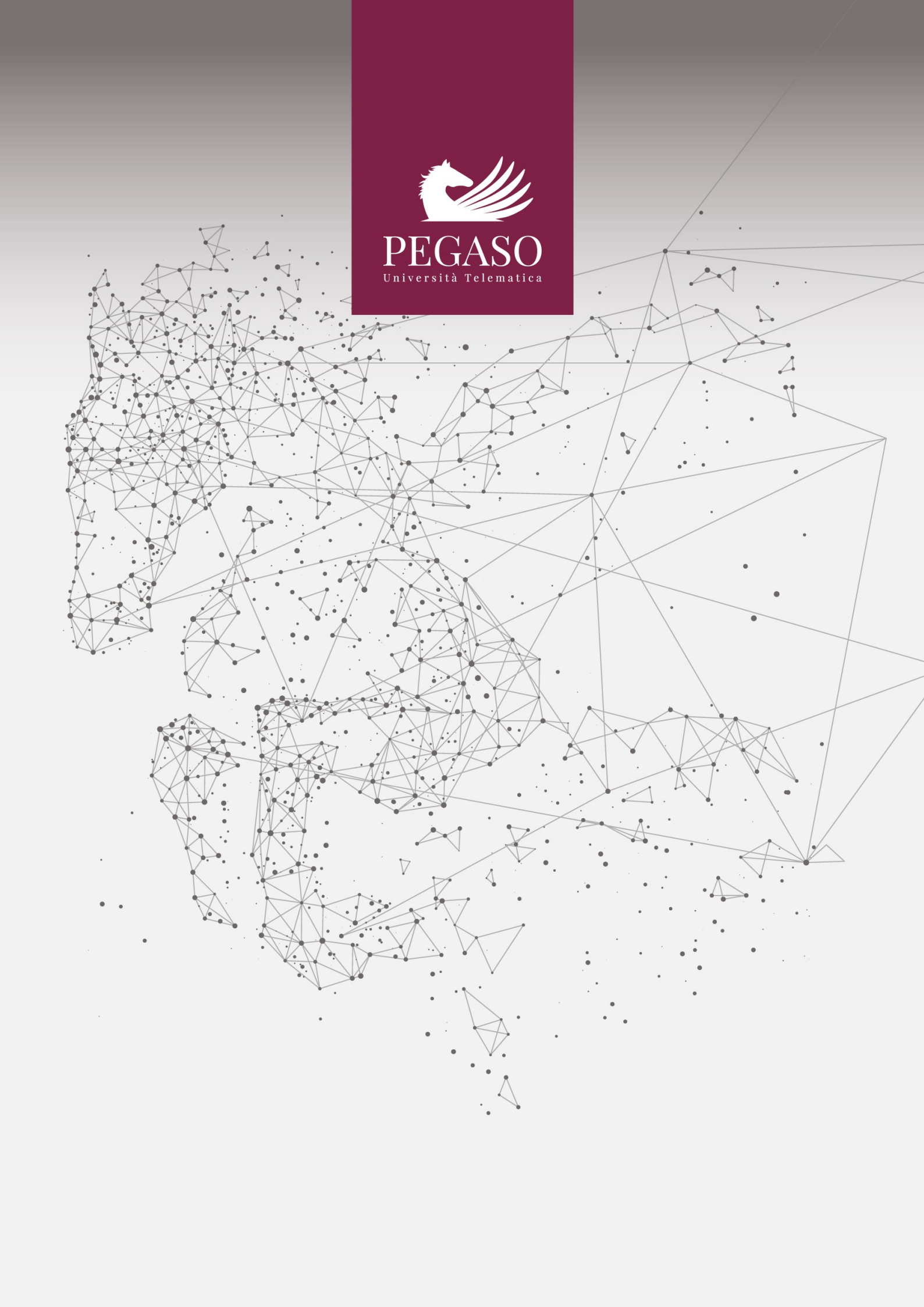

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ISTRUZIONE DI CONFRONTO CON OPERANDO IMMEDIATO SLTI 3

2. ISTRUZIONI LOGICHE CON OPERANDO IMMEDIATO ANDI E ORI 9

3. COSTANTI A 32 BIT 11

BIBLIOGRAFIA 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Istruzione di confronto con operando immediato slti

Oltre al confronto tra variabili, nella programmazione in Linguaggio ad Alto Livello è anche molto utilizzato il confronto tra una variabile ed una costante.

L'Assembly MIPS include istruzioni di confronto ***s**et on **l**ess **t**hen* (imposta a 1 se minore) in cui uno degli operandi è il contenuto di un registro e l'altro è una costante fornita direttamente all'interno dell'istruzione.

Le regole sintattiche Assembly, il Formato e la Modalità di Indirizzamento sono analoghi all'istruzione di addizione *addi,* e vengono richiamati qui di seguito utilizzando un esempio.

L'istruzione **Assembly MIPS** con Codice operativo **slti** confronta il valore di due operandi di cui uno dato da una costante con segno, e produce un risultato che rappresenta l'esito del confronto memorizzato in un registro. La forma sintattica è:

$$\text{slti} \quad \$\text{t1}, \ \$\text{s7}, \ -13$$

l'esecuzione dell'istruzione pone il valore 1 nel registro $t1 se è *vero* che il contenuto del registro $\$\text{s7}$ è minore della costante -13, altrimenti (se l'esito del confronto è *falso*) pone il valore 0 in $\$\text{t1}$.

Vincoli sintattici:

- **Codice Operativo slti**, ***s**et on **l**ess **t**hen immediate* (imposta a 1 se minore della costante immediata),
- **indirizzo del registro DESTINAZIONE**, nell'esempio $\$\text{t1}$:

  contenuto di $\$\text{t1}$ è il valore 1 se è *vero* che contenuto di $\$\text{s7} < -13$,

  contenuto di $\$\text{t1}$ è il valore 0 se è *falso* che contenuto di $\$\text{s7} < -13$.
- **indirizzo del primo operando**, nell'esempio $\$\text{s7}$,
- **costante con segno** che fornisce il secondo operando da confrontare, nell'esempio -13.

L'istruzione che traduce in **Linguaggio Macchina MIPS** l'istruzione Assembly ***set on less then immediato*** utilizza:

- Formato di Tipo I,
- Indirizzamento immediato.

Il Formato di Tipo I deve la sua denominazione al fatto che un dato necessario all'esecuzione è fornito all'interno dell'istruzione, quindi la lettera I è l'iniziale della parola *immediato*.

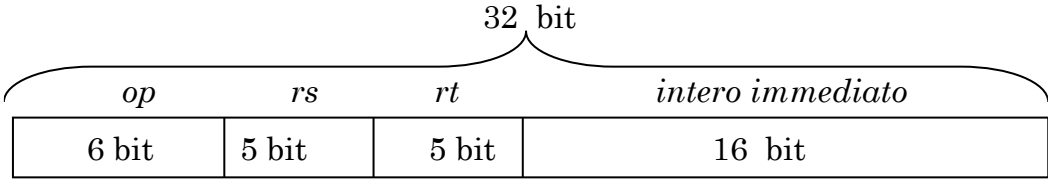
Il **Formato di Tipo I** suddivide i 32 bit della sequenza che rappresenta l'istruzione in 4 campi di lunghezze e posizioni fissate. Il seguente schema indica la **partizione** concettuale (non fisica) in sottosequenze della sequenza di 32 bit dell'istruzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I campi sono denotati con dei nomi simbolici per poterne descrivere l'utilizzazione per le varie istruzioni che usano tale Formato:

FORMATO di Tipo I



op: 6 bit, campo del *Codice Operativo* presente in tutte le istruzioni. indica l'operazione da eseguire ed il Formato dell'istruzione;

rs: 5 bit, indirizzo di registro;

rt: 5 bit, indirizzo di registro;

intero immediato: 16 bit, intero in *Notazione in complemento a* 2.

Il campo dei 6 bit più significativi, presente in tutti i Formati, contiene la sequenza binaria che fornisce il *Codice Operativo* (*op*) che rappresenta l'azione associata all'istruzione. Da questo valore la macchina comprende anche il Tipo di Formato dell'istruzione, e quindi la relativa suddivisione in campi.

I due campi di 5 bit *rs* ed *rt* contengono indirizzi di registri.

I valori contenuti in tali registri sono utilizzati in modi diversi a seconda dell'istruzione.

Il campo di 16 bit contiene un numero intero rappresentato nella *Notazione in complemento a* 2.

Nel caso dell'istruzione ***slti*** il ***Codice Operativo*** è dato dal **valore decimale 10** la cui rappresentazione binaria è 001010.

La rappresentazione delle informazioni fornite dall'istruzione in Linguaggio Macchina è completata dall'operando immediato, e dagli indirizzi relativi al registro del secondo operando e del registro destinazione che contiene il risultato.

Considerando l'esempio di istruzione:

```
slti  $t1, $s7, –13
```

l'operando immediato ha il valore –13, il secondo operando è contenuto nel registro di indirizzo $s7, e il registro destinazione che contiene il risultato ha indirizzo $t1.

Ogni indirizzo è un intero positivo rappresentato da sequenze di 5 bit. Il Formato di Tipo I mette a disposizione 2 campi di lunghezza 5 utilizzabili a questo scopo.

In base alla tabella delle corrispondenze tra notazione simbolica e valore dell'indirizzo, gli indirizzi dell'operando $s7 e del registro destinazione $t1 che contiene il risultato sono dati dai valori:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

$$\$s7 \longleftrightarrow 23 \longleftrightarrow 10111$$
$$\$t1 \longleftrightarrow 9 \longleftrightarrow 01001.$$

Le due sequenze binarie che forniscono il valore degli indirizzi occupano i campi di 5 bit in ordine inverso rispetto all'Assembly, come mostrato nel seguente schema:

slti $t1, $s7, –13

op	*rs*	*rt*	*intero immediato*
10	23	9	–13

la cui rappresentazione binaria è:

001010	10111	01001	1111111111110011

e quindi la sequenza dei 32 bit in Linguaggio Macchina è:

00101010111010011111111111110011

L'istruzione contiene in maniera completa e non ambigua tutte le informazioni necessarie alla sua esecuzione. Un aspetto importante riguarda il come utilizzare le informazioni fornite dai campi del Formato per il reperimento dei dati coinvolti nell'esecuzione. Questa informazione è ottenuta mediante la Modalità di Indirizzamento associata all'istruzione.

Il Formato di Tipo I è utilizzato con diverse Modalità di Indirizzamento. Nel caso dell'istruzione ***slti*** la regola utilizzata è:

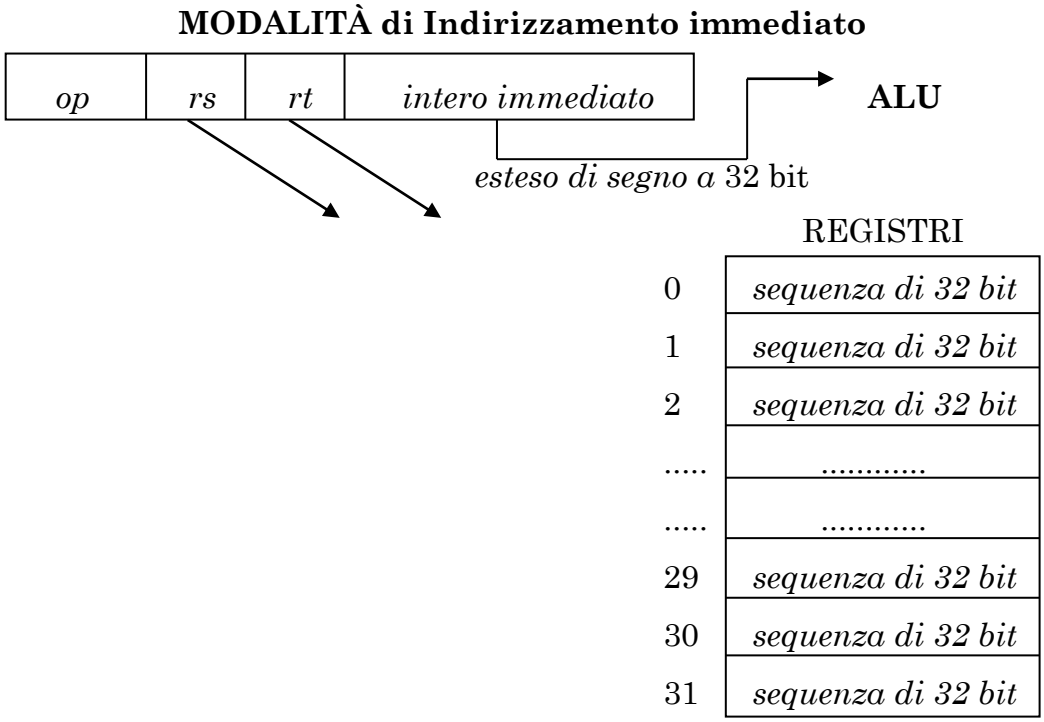
INDIRIZZAMENTO immediato

Il campo di 16 bit del Formato di Tipo I dell'istruzione fornisce uno dei due operandi in Notazione in complemento a 2 rappresentato a 32 bit con *estensione del segno*.

Gli indirizzi dei registri che contengono il secondo operando ed il risultato sono contenuti nei due campi di 5 bit di tale Formato.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seguente figura mostra una rappresentazione schematica del significato di tale Modalità di Indirizzamento:



La limitazione a 16 bit per la rappresentazione dell'operando costante nel *campo immediato* ha due conseguenze:

- Poiché il confronto tra gli operandi (per stabilire il valore di verità della relazione *minore*) con il calcolo del risultato è effettuato dall'**Unità Aritmetico-Logica** (ALU) che esegue operazioni su operandi input a 32 bit, l'Hardware determina la rappresentazione equivalente con 32 bit del valore della costante nel campo immediato di 16 bit effettuando una *estensione del segno* (aggiungendo a sinistra della sequenza i bit mancanti con valore uguale a quello del bit di segno). In base alle proprietà della Notazione in complemento a 2, l'*estensione del segno* non cambia il valore rappresentato.
- Per stabilire i valori che l'operando costante può assumere ricordiamo che in Notazione in complemento a 2 l'intervallo dei valori rappresentabili con 16 bit è compreso tra il valore **minimo negativo** $-2^{16-1} = -2^{15}$, ed il **massimo positivo** $2^{16-1}-1 = 2^{15}-1$, quindi questi valori sono inclusi nell'intervallo dei valori rappresentabili con 32 bit. Il numero di valori forniti dall'operando immediato non cambia, anche se la lunghezza della sequenza viene aumentata a 32 bit con l'*estensione del segno*, perché dipende sempre dai 16 bit del campo immediato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO

Consideriamo l'istruzione:

$$\text{slti} \quad \$t5, \ \$t1, \ -1$$

i 16 bit del *campo immediato* che rappresentano la costante -1 in Notazione in complemento a 2 sono dati dalla sequenza:

$$1111111111111111$$

effettuando l'*estensione del segno* si aggiungono 16 bit più significativi con valore uguale al bit di segno, che in questo caso è 1, ottenendo la seguente sequenza di 32 bit che costituisce l'operando costante:

$$11111111111111111111111111111111$$

che rappresenta ancora il valore -1.

L'esecuzione dell'istruzione valuta la relazione d'ordine < tra il contenuto del registro $t1 (considerato un intero con segno) e l'operando costante rappresentato con 32 bit.

Anche l'istruzione *slti*, come l'istruzione *slt*, viene utilizzata insieme alle istruzioni di salto condizionato per gestire le condizioni logiche nelle istruzioni di controllo del flusso di esecuzione.

ESEMPIO Consideriamo le seguenti istruzioni in Linguaggio C:

```
if ( a < –1 ) {
    min = 0 ;
}
else {
    min = a + 2 ;
}
a = 50 ;
```

***salta** a ELSE*
se a < –1 è ***falsa***

Come *condizione logica* dell'istruzione *if-else* viene valuta la relazione $a < -1$.

Se la condizione è *vera* si effettua l'assegnamento min = 0, se è *falsa* si effettua l'assegnamento min = a + 2. In ogni caso l'istruzione successiva è l'assegnamento a = 50.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Assegnando alle variabili i seguenti registri:

a	min
$t1	$s0

una traduzione in Assembly MIPS è data dalle istruzioni:

```
slti  $t5, $t1, –1          # pone il contenuto di $t5 = 1
                            # se a < –1 è vera, altrimenti lo pone uguale a 0
beq   $t5, $zero, ELSE      #  salta a ELSE
                            # se contenuto di $t5 == 0 è vera, ⟺
                            # se a < –1 è falsa
```

```
        add  $s0, $zero, $zero      #  min = 0
        j  END_IF                   #  esce da if-else
ELSE:   addi  $s0, $t1, 2           #  min = a + 2
END_IF: addi  $t1, $zero, 50        #  a = 50.
```

Si può notare che, per mantenere l'ordine dei casi con *else* come secondo, **nella traduzione Assembly** viene utilizzata l'istruzione di salto condizionato ***beq*** per verificare se il risultato del confronto $a<-1$ eseguito dall'istruzione ***slti*** risulta falso.

Quindi la decisione se effettuare il salto al caso *else* avviene in due passi:

- prima l'istruzione *slti* esegue il confronto in base alla relazione d'ordine *minore* e memorizza il risultato nel registro $t5,
- poi l'istruzione beq effettua il salto se il contenuto di $t5 è 0, che equivale al risultato del confronto falso.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Istruzioni logiche con operando immediato andi e ori

Le istruzioni Assembly AND e OR hanno solo operandi *unsigned* poiché effettuano operazioni *bit a bit*, e anche le istruzioni con operando immediato *andi* e *ori* hanno solo operandi *unsigned*.

Ne segue che il completamento a 32 bit degli operandi immediati rappresentati dai 16 bit del *campo immediato* viene effettuato ponendo i 16 bit più significativi uguali a 0.

Le istruzioni del **Linguaggio Assembly MIPS** hanno, rispettivamente, Codice Operativo **andi** e **ori** con regole sintattiche analoghe a quelle delle istruzioni con operandi immediati *addi* e *slti*:

ESEMPIO
```
andi  $s0,  $t1,  15
ori   $s0,  $t1,  15
```

eseguono, rispettivamente, AND e OR *bit a bit* del contenuto del registro $t1 con la sequenza di 32 bit che rappresenta la costante 15. Il risultato viene scritto nel registro destinazione $s0.

Le corrispondenti istruzioni in **Linguaggio Macchina** hanno Formato e Modalità di Indirizzamento alle istruzioni *addi* e *slti*:

- Formato di Tipo I,
- Indirizzamento immediato.

con i valori dei 6 bit del campo *Codice Operativo* riportati nella seguente tabella:

	6 bit del ***Codice operativo***	
istruzione	decimale	binario
andi	12	001100
ori	13	001101

Invece, una **differenza significativa** è data dal completamento a 32 bit dei 16 bit del *campo immediato* che viene effettuato ponendo sempre a 0 i bit aggiuntivi perché le istruzioni logiche hanno operandi *unsigned*.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nell'esempio precedente la sequenza di 16 bit che rappresenta il valore dell'operando immediato 15 nelle istruzioni *andi* e *ori*:

0000000000001111

viene completata aggiungendo 16 bit più significativi uguali a 0 ottenendo così la seguente sequenza di 32 bit che costituisce l'operando costante delle istruzioni:

0000000000000000**0000000000001111**.

Considerando come contenuto del registro di indirizzo $t1 la sequenza di 32 bit indicata qui di seguito, l'esecuzione delle operazioni *bit a bit* AND e OR fornisce i seguenti risultati:

operando in $t1	11110000111100001111111111111100	AND
costante 15	00000000000000000000000000001111	=
risultato in $s0	00000000000000000000000000001100	
contenuto di $t1	11110000111100001111111111111100	OR
costante 15	00000000000000000000000000001111	=
risultato in $s0	11110000111100001111111111111111	

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Costanti a 32 bit

Una scelta di base dell'Architettura MIPS che consente di semplificare l'Hardware è fissare la lunghezza dell'istruzione in Linguaggio Macchina a 32 bit.

Poiché il campo Codice Operativo di 6 bit è presente in tutte le istruzioni, questa scelta pone delle limitazioni alla possibilità di fornire informazioni all'interno dell'istruzione, sia relative agli indirizzi, sia relative ai valori di operandi costanti che sono rappresentati a loro volta da sequenze di 32 bit.

Le Modalità di Indirizzamento adottate per le istruzioni di salto e di trasferimento dati consentono di ricavare i valori degli indirizzi con opportune regole di calcolo.

Le istruzioni con un operando dato da una costante numerica utilizzano il Formato di Tipo I, che consente di rappresentare il valore di questo operando nei 16 bit del *campo immediato*.

In base alle regole della Notazione in complemento a 2, l'intervallo dei valori rappresentabili con questi 16 bit è compreso tra il minimo -2^{15} ed il massimo $2^{15}-1$. Essendo:

$$2^{15} = 2^5 2^{10} = 32\text{x}1024 = 32768$$

questo intervallo di valori risulta molto limitato rispetto all'intervallo di numeri interi rappresentabili con 32 bit costituito da circa 4 miliardi di valori, che vanno dal minimo -2^{31} ed il massimo $2^{31}-1$. Non avendo la possibilità di rappresentare i 32 bit di un operando costituito da una costante intera all'interno dell'istruzione, per porre un tale valore in un registro occorre utilizzare un'opportuna combinazione delle istruzioni disponibili.

Il seguente esempio mostra una possibile traduzione in Assembly MIPS di una istruzione di assegnamento ad un registro del valore di una costante non rappresentabile con 16 bit.

ESEMPIO Consideriamo l'assegnamento in Linguaggio C:

$$\text{val} = 65539 \ ;$$

assegnando alla variabile val il seguente registro:

$$\text{val} \longleftrightarrow \$\text{s}0$$

Non si può tradurre l'assegnamento ponendo il valore della costante 65539 nel registro $\$\text{s}0$ mediante l'usuale l'istruzione *addi* con Formato di Tipo I perché il numero 65539 in Notazione in Complemento a 2 è rappresentato della sequenza di 32 bit:

PEGASO
Università Telematica

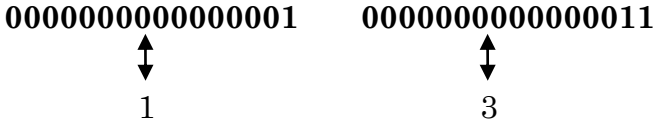
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

$$00000000000000010000000000000011 =$$

$$= 2^{16} + 2 + 1 = 2^6 2^{10} + 3 = 64\text{x}1024 + 3 = 65536 + 3 = 65539$$

che ha 17 cifre significative (sottolineate), mentre il Formato di Tipo I ha solo 16 bit nel *campo immediato* destinato all'operando costante.

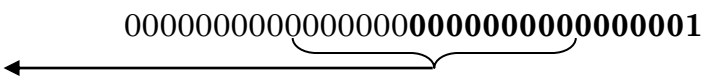
Una soluzione di tipo combinatorio si può ottenere suddividendo la sequenza di 32 bit che rappresenta il numero 65539 in due sottosequenze di 16 bit, che corrispondono rispettivamente ai valori decimali 1 e 3:



Poiché le sottosequenze sono di 16 bit, è possibile utilizzare istruzioni Assembly MIPS con operandi immediati per porre nei 16 bit più significativi di $s0 la sottosequenza con valore 1, e nei 16 bit meno significativi la sottosequenza con valore 3.
Una possibile traduzione è data da:

```
ori $s0, $zero, 1      # pone il valore 1 nel registro $s0
sll  $s0, $s0, 16      # shift di $s0 di 16 posizioni a sinistra
ori $s0, $s0, 3        # OR bit a bit tra il contenuto di $s0 e il valore 3
```

L'esecuzione della prima istruzione *ori* pone nei 16 bit meno significativi del registro $s0 la costante 1 e valori 0 nei rimanenti (il completamento a 32 bit della costante nel campo *intero immediato* di 16 bit viene fatto ponendo 0 nei bit mancanti più significativi):



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il successivo *shift* a sinistra sposta di 16 posizioni i bit della sequenza, così i 16 bit meno significativi diventano i più significativi, ponendo uguali a 0 le posizioni lasciate vuote. A causa dello *shift* si perdono i bit aggiunti con l'*estensione del segno* (che quindi non influenza il risultato) e il contenuto di $s0 diventa:

00000000000000010000000000000000

Anche nella seconda istruzione *ori*, i 16 bit della rappresentazione dell'operando immediato 3 vengono completati aggiungendo 16 bit più significativi tutti uguali a 0, ottenendo così la sequenza di 32 bit che rappresenta l'operando costante:

0000000000000000**0000000000000011**.

L'esecuzione dell'OR *bit a bit* tra questo operando costante ed il contenuto del registro $s0 ha come risultato la sequenza di 32 bit che rappresenta il valore 65539, e che viene scritta nel registro destinazione $s0 ottenendo così l'assegnamento val = 65539:

00000000000000010000000000000000	OR	$s0
0000000000000000**0000000000000011**	=	3
0000000000000000 0000000000000011	⟷	65539

Le seguenti Tabelle riassumono per il sottoinsieme di istruzioni presentate le regole sintattiche del Linguaggio Assembly e del Linguaggio Macchina MIPS, e la relativa Modalità di Indirizzamento.

I valori dei campi del Formato in Linguaggio Macchina sono espressi mediante la notazione decimale per i valori numerici, e la notazione simbolica per gli indirizzi dei registri del processore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

istruzione	*Formato Tipo*	*Modalità di Indirizzamento*
add	R	tramite registro
sub	R	tramite registro
and	R	tramite registro
or	R	tramite registro
slt	R	tramite registro
sll	R	tramite registro
srl	R	tramite registro
lw	I	tramite Base e Offset
sw	I	tramite Base e Offset
beq	I	relativo al *Program Counter*
bne	I	relativo al *Program Counter*
j	J	pseudodiretto
jal	J	pseudodiretto
jr	R	tramite registro
addu	R	tramite registro
subu	R	tramite registro
sltu	R	tramite registro
addi	I	immediato
slti	I	immediato
andi	I	immediato
ori	I	immediato

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6 bit	5 bit	5 bit	5 bit	5 bit	6 bit		esempio
istruzione	*Tipo*	*op*	*rs*	*rt*	*rd*	*shamt*	*funct*	
add	R	0	$s1	$s2	$t0	0	32	add $t0, $s1, $s2
sub	R	0	$s1	$s2	$t0	0	34	sub $t0, $s1, $s2
and	R	0	$s1	$s2	$t0	0	36	and $t0, $s1, $s2
or	R	0	$s1	$s2	$t0	0	37	or $t0, $s1, $s2
slt	R	0	$s1	$s2	$t0	0	42	slt $t0, $s1, $s2
sll	R	0	0	$s2	$t0	7	0	sll $t0, $s2, 7
srl	R	0	0	$s2	$t0	7	2	srl $t0, $s2, 7
lw	I	35	$s3	$t0	–9			lw $t0, –9 ($s3)
sw	I	43	$s3	$t0	–9			sw $t0, –9 ($s3)
beq	I	4	$t0	$s3	istruzioni da saltare			beq $t0, $s3, *nome*
bne	I	5	$t0	$s3	istruzioni da saltare			bne $t0, $s3, *nome*
j	J	2	indirizzo di salto incompleto					j *nome*
jal	J	3	indirizzo di salto incompleto					jal *nome*
jr	R	0	$ra	0	0	0	8	jr $ra
addu	R	0	$s1	$s2	$t0	0	33	addu $t0, $s1, $s2
subu	R	0	$s1	$s2	$t0	0	35	subu $t0, $s1, $s2
sltu	R	0	$s1	$s2	$t0	0	43	sltu $t0, $s1, $s2
addi	I	8	$s3	$t0	–9			addi $t0, $s3,–9
slti	I	10	$s3	$t0	–9			slti $t0, $s3,–9
andi	I	12	$s3	$t0	9			andi $t0, $s3, 9
ori	I	13	$s3	$t0	9			ori $t0, $s3, 9

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).